

Ipari képfeldolgozás projektmunka

I. Mérföldkő

Készítette: Takács Ábel

Járműmozgás-elemző Rendszer – Algoritmus- és Költségterv

1. Projektáttekintés

A rendszer célja videófelvételeken közlekedő járművek automatikus detektálása, mozgási útvonaluk rekonstruálása, a forgalmi trendek meghatározása, valamint a trendektől eltérő (anomális) mozgásminták azonosítása. A megvalósítás Python nyelven, OpenCV könyvtár felhasználásával történik.

2. Algoritmus terv

2.1 A feldolgozás folyamata

```
Videóbemenet → Előfeldolgozás → Járműdetektálás → Járműkövetés  
→ Útvonalrekonstrukció → Trendanalízis → Anomáliadetekció → Kimenet
```

[2.1] - A feldolgozás menetét szemléltető pipeline

2.2 Előfeldolgozás

Cél: A nyers videóképkockák zajszűrése és normalizálása a detektálás pontosságának javítása érdekében.

Lépések:

- Képkocka beolvasása (cv2.VideoCapture)
- Szürkeárnyaltos konverzió (cv2.cvtColor)
- Gauss-simítás a zajszűrés eléréséhez (cv2.GaussianBlur)

Pszeudokód:

FOR every frame f_t IN video:

$gray_t = cvtColor(f_t, BGR2GRAY)$

$smooth_t = GaussianBlur(gray_t, kernel=(5, 5), o=1.5)$ // Finomhangolható

END FOR

2.3 Járműdetektálás

Módszer: MOG2 (Mixture of Gaussian v2) háttérsubsztrakció, mely adaptívan képes modellezni a háttérre. Ezzel elkészíthető egy modell a háttérre, amennyiben egy pixel/kernel-átlag szignifikáns eltérést mutat a modelltől potenciális járműnek tekinthető.

Peszudokód:

```

FOR every smooth_t:
    mask_t = mog2.apply(smooth_t)
    mask_t = morphologyEx(mask_t, MORPH_CLOSE, kernel) // Lyukak tömitése
    mask_t = morphologyEx(mask_t, MORPH_OPEN, kernel) // Zajsztűrés
    contours = findContours(mask_t) // Körvonalak detektálása

    FOR every c IN contours:
        IF area(c) > MIN_AREA: // Miniális jármű területe a képen
            bbox = boundingRect(c) // A körvonal négyszöggé alakítása
            detections_t.append(bbox)
    END FOR
END FOR

```

2.4 Járműkövetés

Cél: Azonos jármű detektálása egymást követő képkockákon és pályájának folyamatos nyomonkövetése.

2.4.1 Kalman-szűrő - Követés

Minden detektált járműhöz fenntartunk egy Kalman-szűrőt, melynek állapotvektora:

$$\mathbf{x} = [u, v, u', v']^T$$

ahol (u, v) a bbox középpontja, (u', v') pedig a sebességkomponensek.

Predikció (állapotátmenet):

$$\mathbf{x}_{t|t-1} = \mathbf{F}\mathbf{x}_{t-1|t-1}$$

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Frissítés:

$$\text{Kalman erősítés számítása: } \mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}^T (\mathbf{H} \mathbf{P}_{t|t-1} \mathbf{H}^T + \mathbf{R})^{-1}$$

ahol $\mathbf{H}$ a megfigyelési mátrix, $\mathbf{P}$ a becsült kovariancia, $\mathbf{R}$ a mérési zaj.

$$\text{Frissített állapot számítása: } \mathbf{x}_{t|t} = \mathbf{x}_{t|t-1} + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H} \mathbf{x}_{t|t-1})$$

ahol $(\mathbf{z}_t - \mathbf{H} \mathbf{x}_{t|t-1})$ az innovációs vektor (a mérés és a becslés eltérése), $\mathbf{z}_t$ az állapotbecslés.

2.4.2 Magyar algoritmus – Hozzárendelés

A detektált boxok és az aktív követések összerendelése IoU (Intersection over Union) alapú költségmátrix segítségével:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Pszeudokód:

```

predicted_pos = [kalman-predict() FOR k IN active_trackers]
cost_m = 1 - IoU_matrix(predicted_pos, detections_t)
row_ind, col_ind = hungarian_alg(cost_m)

```

FOR every (i, j) párosításban:

IF $cost_m[i][j] < THRESHOLD$:

$active_trackers[i].kalman_update(detections_t[j])$

ELSE:

$új\ tracker\ létrehozása\ detections_t[j]-hez$

END FOR

$eltűnt\ tracker-ek\ törlése\ (min\ N\ képkockán\ át\ nem\ detektált)$

2.5 Útvonal rekonstrukció:

Minden jármű útvonala a detektált középpontok időrendezett sora. Ezt az útvonalat finomhangolási lépésként lehet simítani.

2.6 Trendanalízis:

Cél: A jellemző mozgásminták (útvonalak, sebességek, irányok) meghatározása a teljes felvételen.

Módszer: Az útvonalak reprezentációja, majd K-means klaszterezés.

A detektált útvonalakat egységes hosszúságú Feature Vektorokká alakítjuk, majd ezeket a vektorokat klaszterezzük. Mivel nem biztosított, hogy az összes detektált jármű ugyanannyi képkockán látszódik, így ez egy szükséges egységesítési lépés.

Lineáris interpolációval minden útvonalat N egyenlő közű pontra mintavételezünk (N=20 egy tipikus választás), ezzel megkapjuk a Feature Vektort. A kapott vektor normalizálással finomhangolható.

A klaszterek számát a könyök-módszerrel meghatározhatjuk, vagyis az összesített négyzetes eltérés görbéjének töréspontja alapján. A cél, hogy az azonos csoportokba kerülő útvonalak minél hasonlóbbak legyenek egymáshoz.

$$WCSS(K) = \sum_{j=1}^K \sum_{\mathbf{f}_k \in S_j} \|\mathbf{f}_k - \mathbf{C}_j\|^2$$

Ahol $\mathbf{C}_j$ a j-edik klaszter centroidja (az összes klaszterbe tartozó útvonal átlaga), S_j a klaszterbe tartozó útvonalak halmaza, $\mathbf{f}_k$ pedig az elemzett útvonal Feature Vektora.

Az algoritmus lépései:

- 1) Kiválasztunk K centroidot véletlenszerűen
- 2) Minden útvonalat a legközelebbi centroidhoz rendelünk
- 3) A centroidokat újraszámítjuk a hozzájuk rendelt útvonallal
- 4) Ha a centroidok nem változnak megállunk, amúgy 2-es lépéstől folytatjuk amíg nem konvergál az érték

Az optimális K meghatározása (könyök módszer):

Mivel nem tudhatjuk hány tipikus útvonal létezik, így futtatjuk a K-means algoritmust $K=1, 2, \dots, K_{max}$ értékekre és ábrázoljuk a WCSS értékeket. Az optimális K érték ott van, ahol d^2WCSS maximális.

2.7 Anomáliadetektálás

Cél: A meghatározott trendektől szignifikánsan eltérő mozgások beazonosítása.

Módszer: Mahalanobis-távolság alapú küszöbölés.

Egy Tk útvonal anomália-pontszáma a legközelebbi klaszterközéptől való eltérés Mahalanobis-távolságban számolva.

$$D_M(\mathbf{f}_k, \mathbf{C}_j) = \sqrt{(\mathbf{f}_k - \mathbf{C}_j)^T \cdot \Sigma_j^{-1} \cdot (\mathbf{f}_k - \mathbf{C}_j)}$$

Ahol Σ_j a j-edik klaszter kovarianciamátrixa:

$$\Sigma_j = \frac{1}{|S_j|} \sum_{\mathbf{f}_k \in S_j} (\mathbf{f}_k - \mathbf{C}_j)(\mathbf{f}_k - \mathbf{C}_j)^T$$

A döntési szabály, hogy egy $\mathbf{f}_k$ útvonal akkor minősül anomáliának, amennyiben a Mahalanobis-távolságának minimuma meghalad egy küszöbértéket. Ennek az értéknek a meghatározása leghatékonyabban a távolságok eloszlásának 95. percentilise alapján oldható meg.

Pszeudokód:

FOR every T_k útvonal:

$dist = [Mahalanobis(T_k, c_j, sum_j) \text{ FOR } j \text{ IN clusters}]$

$min_dist = \min(dist)$

IF $min_dist > THETA$: // Az eltérés küszöbértéke

T_k megjelölése anomáliaként

naplózás (timestamp, pozíció, eltérés mértéke)

END FOR

3 Költségterv:

3.1 Fejlesztési idő:

Folyamat	Időigény
Környezet beállítása és adatbeszerzés	4 óra
Előfeldolgozás és detektálás	16 óra
Kalman-szűrő + követési modul	16 óra
Útvonalrekonstrukció	8 óra
Trendanalízis (klaszterezés)	10 óra
Anomália detekció	10 óra
Vizualizáció	8 óra
Tesztelés és finomhangolás	16 óra
Összesen	~88 óra

3.2 Hardverköltiségek

Eszköz	Megjegyzés	Becsült ár
IP kamera	4MP, Color, FullHD, 30fps, éjjellátó	50.000 Ft
PoE switch	Tápellátás hálózaton keresztül	20.000 Ft
Szerelési anyagok	~30 méteres kiépítés	10.000 Ft
Szerverköltiségek	Folyamatos videofeldolgozáshoz CPU/GPU és tárhely igény	450.000 Ft
Alaplap+CPU	16 mag, magas órajel	60.000 Ft + 120.000 Ft
RAM	Min. 32 GB, videópuffer	80.000 Ft
GPU	CUDA, OpenCV GPU támogatás	130.000 Ft
SSD	Min. 500GB	30.000 Ft
Egyéb alkatrészek	Táp, Ház stb.	50.000 Ft
NAS tárolóegység	Biztonsági mentések, felvételek hosszútávú tárolása	120.000 Ft
Hálózati infrastruktúra	Router, UPS	50.000 Ft
Összesen		~720.000 Ft

Implementációs részletek és eredmények bemutatása

A teszteléshez felhasznált közlekedésfigyelő kamera videó: <https://www.youtube.com/watch?v=eO19UTm93GQ>.

A projektet tartalmazó git repository: <https://github.com/FreRare/vehicle-movement-detection>

Megvalósítás:

Járműdetektálás:

- Előfeldolgozás Gauss-simítást alkalmazva
- Kontúrok detektálása
- A kontúrt határoló téglalap és középpontjának meghatározása

Járműkövetés:

- Járműkövetéshez szükséges adatstruktúra
- Kálmán szűrő használata a következő képkockán való hely becsléséhez
- Magyar algoritmus a becslés és a valós helyzet összevetéséhez
- Követett járművek frissítése az eddigi adatok és az aktuális képkocka adatai alapján

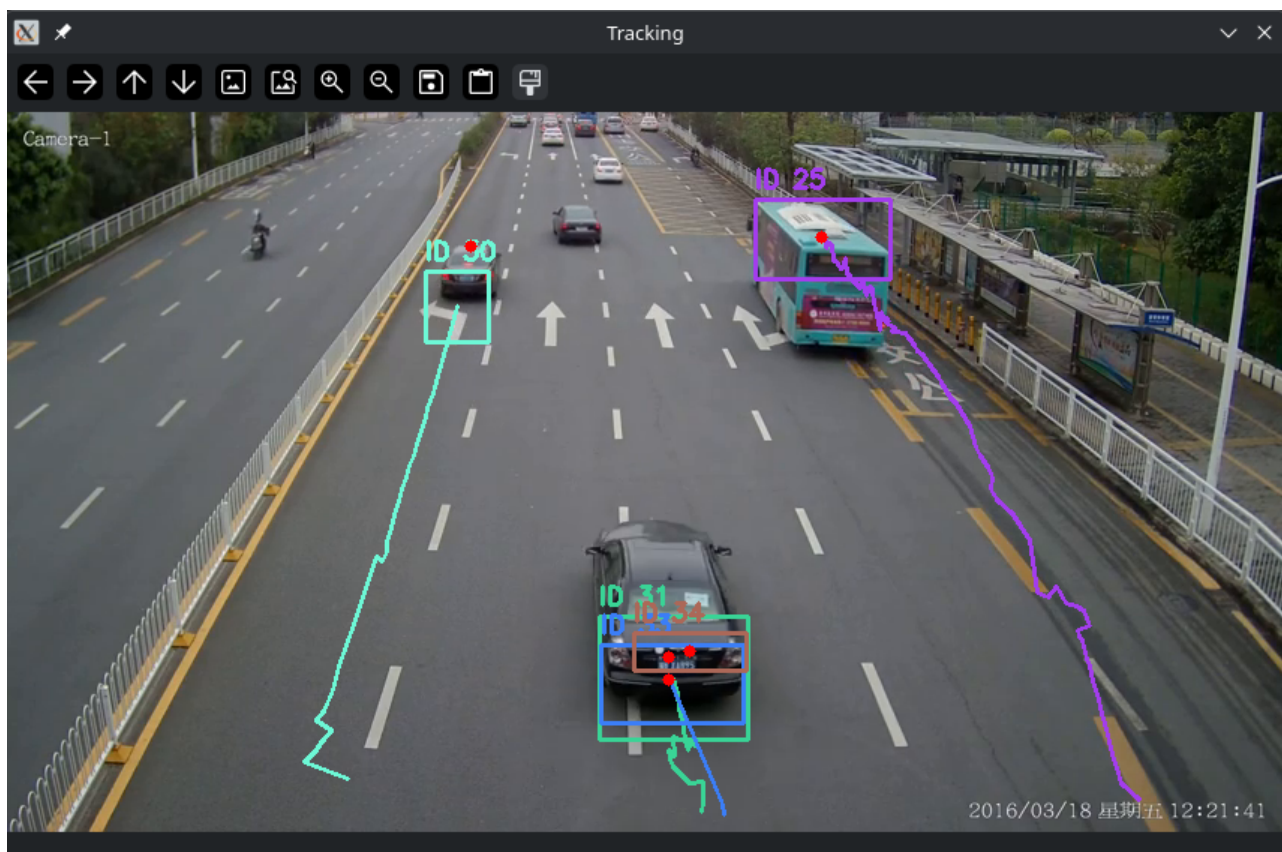
Trendek követése:

- Eddigi összes útvonal tárolása (Jármű archívum)
- A követett járművek útvonalának kiszámítása interpolációval
- Útvonalak Feature Vektorra alakítása
- Feature mátrix felépítése (aktuálisan követett járművek alapján)
- Optimális K érték dinamikus számítása könyök metódussal
- Az aktuális útvonalak klaszterezése (Centroid számítás)
- Átlag sebesség és irány számítása [px/frame, szög]
- Klaszterezett útvonalak kirajzolása
- A videó végén az összes klaszter alapján trendek meghatározása
 - Súlyozott értékkel az irányra és elhelyezkedésre, hogy a közelinek látszó sávokat és egymással szemben haladó autókat is el lehessen határolni.

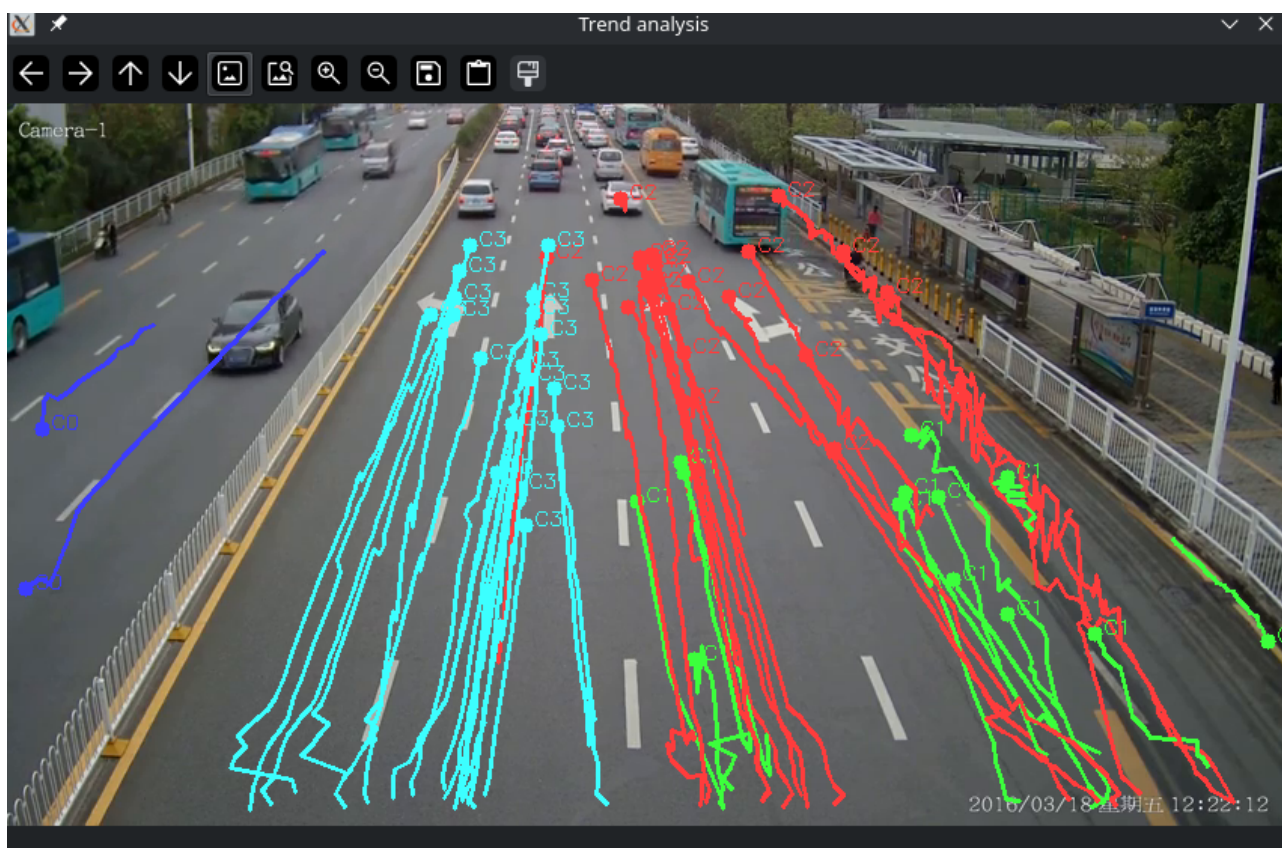
Főprogram:

- Videó beolvasása
- Képkockák egyenkénti feldolgozása az előbb felsorolt algoritmusokkal
- Trend analízis interval konfigurálása (megadja hány képkockánként analizálja az útvonalakat, ha kevés az adat akkor nem tud dolgozni)
- Videó végén, vagy lejátszás közben 'q' billentyű lenyomására az addig feldolgozott adatok alapján teljeskörű trend analízis és ezek összegzése külön képeken
- Második alkalomra bármely billentyű lenyomására kilépés (ha vége a feldolgozásnak akkor is ez a kilépés módja)

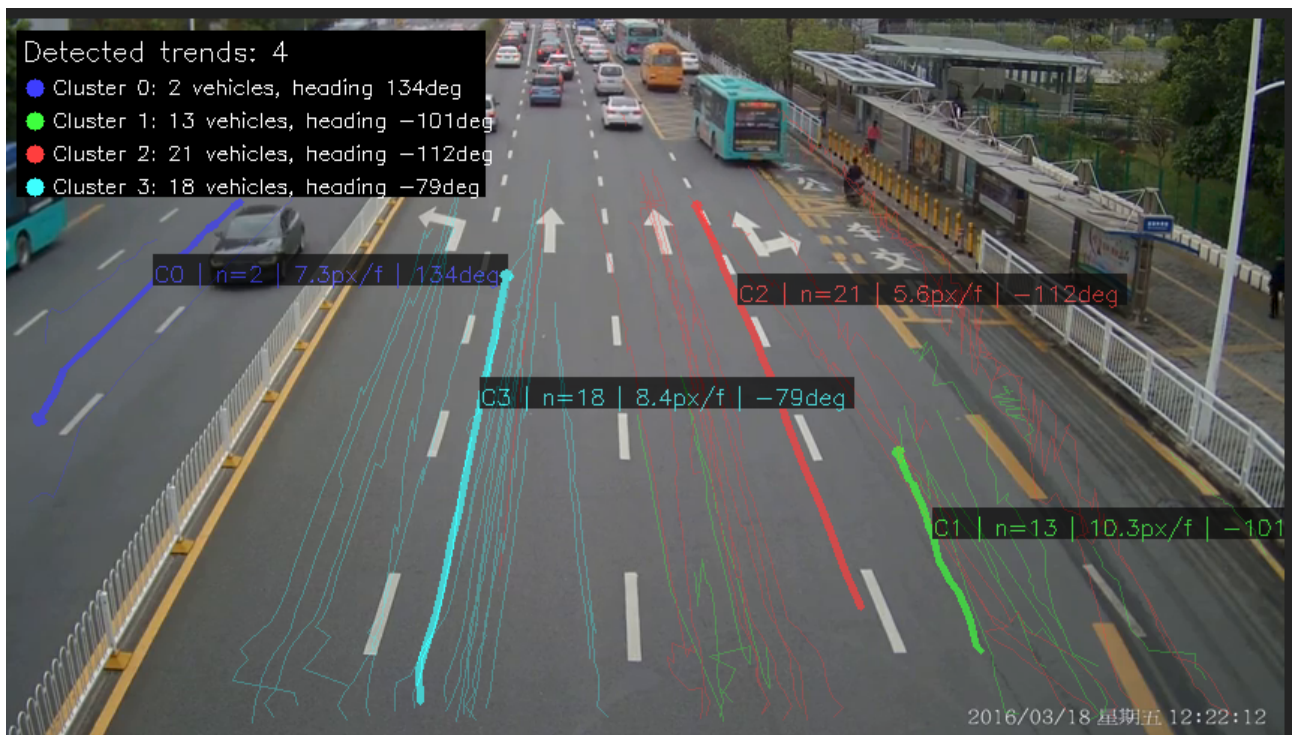
Elért eredmények:



Járművek követése a videón



Az útvonalak trend-kaszterekbe gyűjtve (színek szerint)



Összegző trend analízis a teljes videó alapján

Megjegyzések:

A program repository-jában megtalálható a teszteléshez használt videó, valamint ez a dokumentum is.

Az algoritmus rengetek konfigurálható paraméterrel rendelkezik, melyeket a különféle videókhoz szükséges lehet finomhangolni. Igyekeztem a teszt videóhoz legjobban illő beállítást választani az összegző trend analízis optimalizálásához.

A projekt nem lett végtelenségig tesztelve, előfordulhat, hogy futási hibák lépnek fel, legfőképp a trend analízis modul érzékeny a túl gyors egymást követő analízisekre, hiszen előfordulhat, hogy nincsen elegendő adat az algoritmus paramétereinek meghatározásához (emiatt javasolt legalább 150 de inkább 200 frame feletti intervallt beállítani ehhez).

A projekt rálátást biztosított ezeknek a rendszereknek a finomságára és érzékenységre, sokkal jobban átlátom, miért érdemes gépi tanulási módszerekkel meghatározni a feldolgozási paramétereket.

A feladat komplexitása és az idő szűke miatt a trendektől való eltérések meghatározása funkciót sajnos nem sikerült megvalósítani.